

Unity Save System

Лёгкая система сохранений: JSON-файл в `Application.persistentDataPath`,
версионирование с миграциями, сброс к дефолтам (по одному объекту или все сразу),
поддержка Unity-типов (`Vector2/3`, `Quaternion`, `Color`) через кастомные конвертеры.

Требования

- Unity 2021.3+
- Пакет **Newtonsoft Json**: Package Manager → Add package by name → `com.unity.nuget.newtonsoft-json`

Состав

Файл	Что это
<code>ISavable.cs</code>	Интерфейс для объектов, которые хотят сохраняться
<code>ISaveMigration.cs</code>	Интерфейс одной миграции версии ($N \rightarrow N+1$)
<code>SaveSystem.cs</code>	Ядро: хранение, файл, версии, миграции, сбросы
<code>UnityJsonConverters.cs</code>	Конвертеры <code>Vector2/Vector3/Quaternion/Color</code>
<code>Examples.cs</code>	Примеры: <code>ISavable</code> , миграция, точка входа (можно не копировать)

Принцип работы

Хранилище

Все данные живут в памяти в одном `JObject` — по сути это словарь `ключ → JSON-подде`

`SaveToSaves` пишет в память, на диск всё уходит при `SaveAll()` (и при одиночном сбросе).
Это дёшево — можно сохранять состояние часто, запись файла произойдёт один раз.

Жизненный цикл

Старт игры: `LoadAll()` → читаем файл → миграции → каждый `ISavable.Load(this)`
Во время игры: объекты меняют состояние
Сохранение: `SaveAll()` → каждый `ISavable.Save(this)` → запись файла
Сброс одного: `ResetToDefaults(savable)` → удаляем его ключ → `Load` даёт дефолты → файл
Сброс всех: `ResetToDefaults()` → чистим память, удаляем файл → все `Load` дают дефолт

Каждый `ISavable` объявляет свой уникальный `SaveKey`, сам решает, что хранить, и сам задаёт дефолты в `Load`. Системе не нужно знать о конкретных объектах — она просто хранилище ключей. Дубликат `SaveKey` ловится ошибкой в логе при `Register`.

Формат файла

persistentDataPath/saves.json:

```
{
  "version": 2,
  "data": {
    "wallet": { "Gold": 150, "Gems": 3, "LastPosition": { "x": 0, "y": 1, "z": 0 } }
    "settings": { "MusicVolume": 0.8 }
  }
}
```

Версия лежит в "конверте" снаружи данных — поэтому миграции могут переписывать data как угодно. Запись атомарная: сначала во временный .tmp, потом подмена файла — краш посреди записи не убьёт сейв.

Версии и миграции

Когда меняешь формат сейва (переименовал ключ, поменял структуру State):

1. Подними `SaveSystem.CurrentVersion` (например, $1 \rightarrow 2$).
2. Напиши миграцию с `FromVersion = 1`, которая превращает данные версии 1 в версию 2.

При загрузке миграции применяются цепочкой: файл версии 1 при `CurrentVersion = 3` пройдёт через миграции $1 \rightarrow 2$ и $2 \rightarrow 3$. Миграция работает с сырым `JObject`, а не с типами — старых классов в коде уже может не быть. Если какой-то миграции в цепочке нет, сейв сбрасывается к дефолтам (лучше, чем краш на битых данных).

```
public class Migration_1_to_2 : ISaveMigration
{
    public int FromVersion => 1;

    public void Migrate(JObject data)
    {
        if (data.Remove("money", out var token))
            data["wallet"] = token;
    }
}
```

Конвертеры

Newtonsoft не умеет Unity-типы из коробки: у `Vector3` есть свойство `normalized`, которое возвращает новый `Vector3` — сериализатор уходит в бесконечную рекурсию. Конвертеры из `UnityJsonConverters.cs` пишут такие типы вручную как `{ "x":1, "y":2, "z":3 }`. Все вызовы `SaveToSaves/LoadFromSaves` идут через общий `JsonSerializer` с этими конвертерами. Свои конвертеры (например, для `Vector3Int`) докидываются вторым аргументом конструктора.

Быстрый старт

```
public class PlayerWallet : ISavable
{
    [Serializable]
    private class State { public int Gold; public Vector3 LastPosition; }

    public string SaveKey => "wallet";

    public int Gold { get; private set; }
    public Vector3 LastPosition { get; private set; }

    public void Save(SaveSystem saves)
        => saves.SaveToSaves(SaveKey, new State { Gold = Gold, LastPosition = LastPosition });

    public void Load(SaveSystem saves)
    {
        var s = saves.LoadFromSaves(SaveKey, new State { Gold = 100 }); // дефолты тут
        Gold = s.Gold;
        LastPosition = s.LastPosition;
    }
}
```

```
// точка входа (без DI)
var saves = new SaveSystem(new ISaveMigration[] { new Migration_1_to_2() });
saves.Register(wallet);
saves.LoadAll();
// ... игра ...
saves.SaveAll();

// сбросы
saves.ResetToDefaults(wallet); // только кошелёк, файл переписывается сразу
saves.ResetToDefaults();      // всё, файл удаляется
```

```
// VContainer
builder.Register<SaveSystem>(Lifetime.Singleton);
builder.Register<ISaveMigration, Migration_1_to_2>(Lifetime.Singleton);
// SaveSystem получит IEnumerable<ISaveMigration> автоматически
```

API

Метод	Описание
Register(ISavable) / Unregister(ISavable)	Подписать объект; дубликат SaveKey — ошибка в лог
SaveToSaves<T>(key, value)	Положить значение в хранилище (в память)
LoadFromSaves<T>(key, default)	Достать значение, при отсутствии — дефолт
HasKey(key) / DeleteKey(key)	Проверка/удаление ключа
SaveAll()	Собрать состояние со всех ISavable и записать файл

Метод	Описание
<code>LoadAll()</code>	Прочитать файл, прогнать миграции, раздать состояние
<code>ResetToDefaults(ISavable)</code>	Сброс одного объекта, файл переписывается сразу
<code>ResetToDefaults()</code>	Полный сброс: память + файл, все получают дефолты

Подводные камни

- Пиши в `Application.persistentDataPath`, не в `Application.dataPath` — последний read-only в билдах.
- На Android `OnApplicationQuit` не гарантирован — вызывай `SaveAll()` в `OnApplicationPause(true)`.
- Одиночный сброс переписывает файл текущим содержимым памяти: для остальных ключей на диск уйдёт то, что лежало в `_data` на момент сброса (последние `SaveToSaves`), а не свежесобранное через `Save()` состояние объектов.
- Битый файл при чтении тихо сбрасывается к дефолтам — смотри лог ошибок.